

Memory Management in TinyML

Static Memory Allocation Strategy:

- No `malloc()/free()` to avoid heap fragmentation
- Pre-planned memory layout for long-running applications
- Fixed-size arena passed at initialization
- Immediate error if arena too small

Memory Arena Concept:

```
// Application provides memory arena
constexpr int kTensorArenaSize = 60 * 1024;
uint8_t tensor_arena[kTensorArenaSize];

// Framework uses arena for all allocations
tflite::MicroInterpreter interpreter(model, resolver,
                                     tensor_arena, kTensorArenaSize);
```

uint8_t is a typedef in C and C++ that represents an unsigned 8-bit integer

Benefit: Predictable memory usage, no runtime allocation failures

RAM Usage Analysis

```
// Static allocation approach
uint8_t tensor_arena[50 * 1024]; // 50KB arena

tflite::MicroInterpreter interpreter(
    model, resolver, tensor_arena,
    sizeof(tensor_arena), error_reporter);

// Check if arena is sufficient
if (interpreter.AllocateTensors() != kTfLiteOk) {
    // Arena too small - increase size
}
```

Dynamic Memory Requirements

TensorFlow Lite Memory Arena:

The arena holds temporary activation tensors during inference

Arena size calculation:

- `arena_size = max_simultaneous_tensor_memory`

Typical requirements by model type:

- **Simple_dense_model**: 5-20KB
- **Small_CNN**: 20-100KB
- **Efficient_transformer**: 50-200KB

Application Memory:

- Input/output buffers
- Preprocessing workspace
- Communication buffers
- Business logic variables

Memory-Constrained Model Design

Goal: Fit models in limited flash/SRAM while keeping accuracy usable

Budget Planning

- Know flash size (weights/code) and SRAM size (activations/scratch)
- Track peak live activations, not just parameters
- Use static memory plans; preallocate reuse buffers

Design Techniques

- Layer-wise processing
- Run one layer at a time
- Stream weights from flash
- Reuse a single activation buffer across layers

In-place ops

- Use ReLU and similar in-place to avoid extra tensors
- Minimize temporary buffers

Operator fusion

- Fuse Conv + BN + ReLU
- Reduce intermediate tensors and memory traffic

Quantization

- INT8 weights/activations; 32-bit accumulators if needed

Pruning and sparsity

- Remove low-importance channels/connections
- Exploit sparse formats when supported

Shape control

- Cap feature-map sizes (channels/resolution)
- Avoid long skip connections that keep tensors live

Validation

- Measure peak SRAM during inference on device
- Iterate until within flash/SRAM limits